

# CXLMemSim: Practical Performance Simulation and Characterization of CXL 3.0 Memory Systems

Yiwei Yang  
yyang363@ucsc.edu  
UC Santa Cruz  
Santa Cruz, USA

Shri Vishakh  
Devanand  
vdshri\_b23@el.vjti.ac.in  
Veermata Jijabai  
Technological Institute  
Mumbai, India

Brian Zhao  
bwzhao@ucsc.edu  
UC Santa Cruz  
Santa Cruz, USA

Yusheng Zheng  
yzhen165@ucsc.edu  
UC Santa Cruz  
Santa Cruz, USA

Pooneh Safayenikoo  
psafayen@ucsc.edu  
UC Santa Cruz  
Santa Cruz, USA

Tanvir Ahmed Khan  
tk3070@columbia.edu  
Columbia University  
New York, USA

Andi Quinn  
aquinn1@ucsc.edu  
UC Santa Cruz  
Santa Cruz, USA

## Abstract

Compute Express Link (CXL) 3.0 can turn stranded DRAM into a composable datacenter memory fabric, but hardware is scarce and cycle-accurate simulation is prohibitively slow for full workloads. CXLMemSim is an open-source software CXL memory simulator that keeps computation at native speed and models only CXL-affected memory behavior. Across SPEC CPU2017, Memcached, wrf, vector search, and llama.cpp, CXLMemSim stays within 11–15% of hardware and gem5 while completing a Redis-scale run in 45 minutes instead of gem5’s 73 hours. The public release at SlugLab/CXLMemSim provides PEBS/LBR/eBPF tracing, JSON topology descriptions, and reusable migration/cache policy hooks for CXL design-space exploration.

## CCS Concepts

• **Computer systems organization** → **Cloud computing; Heterogeneous (hybrid) systems**; • **Computing methodologies** → *Modeling and simulation*.

## Keywords

CXL, memory disaggregation, memory systems, simulation, performance modeling

## ACM Reference Format:

Yiwei Yang, Shri Vishakh Devanand, Brian Zhao, Yusheng Zheng, Pooneh Safayenikoo, Tanvir Ahmed Khan, and Andi Quinn. 2026. CXLMemSim: Practical Performance Simulation and Characterization of CXL 3.0 Memory Systems. In *The 35th International Symposium on High-Performance Parallel and Distributed Computing (HPDC ’26)*, July 13–16, 2026, Cleveland, OH, USA. ACM, New York, NY, USA, 3 pages. <https://doi.org/10.1145/3806645.3820069>



This work is licensed under a Creative Commons Attribution 4.0 International License. *HPDC ’26, Cleveland, OH, USA*

© 2026 Copyright held by the owner/author(s).  
ACM ISBN 979-8-4007-2640-8/2026/07  
<https://doi.org/10.1145/3806645.3820069>

| Approach                                 | Strength                                                   | Runtime / limitation                                                                                                                |
|------------------------------------------|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Real CXL hardware<br>NUMA/RDMA emulation | Ground-truth timing<br>Runs full software stacks           | Scarce, expensive, fixed topology<br>Near-native speed but misses CXL coherence, switch, and device-side bottlenecks                |
| gem5-style simulation<br>CXLMemSim       | Detailed timing model<br>Full applications with CXL timing | 73 hours for a Redis-scale run; too slow for policy sweeps [1, 8]<br>45 minutes for the same Redis-scale run; 123× faster than gem5 |

**Table 1: Why existing CXL evaluation paths are impractical for broad topology and policy studies.**

## 1 Motivation

CXL addresses a persistent datacenter inefficiency: servers can be CPU saturated while 20–25% of local DRAM remains unused, stranding expensive memory behind rigid machine boundaries [2, 6, 7, 10]. CXL.mem provides cache-coherent load/store access to device-attached Type-3 memory over PCIe-derived links, and CXL 3.0 adds multi-level switching, fabric coherence, and back invalidation [4, 11]. These features make disaggregated memory plausible for unmodified software, but they also introduce 2–3× higher latency than local DRAM, 70–180 ns additional access delay, 25–30 GB/s per-link bandwidth on current platforms, and extra delay from each switch hop [3, 7].

Table 1 quantifies the core motivation. Existing methods force an uncomfortable tradeoff between realism and iteration speed: real hardware is narrow and fixed, NUMA/RDMA emulation misses CXL-specific behavior [5, 9, 12], and cycle-accurate simulation is too slow for realistic policy exploration. CXLMemSim fills this gap by running full applications while modeling the CXL-specific memory behavior that determines latency, bandwidth, congestion, and coherence cost.

## 2 Design and Open-Source Release

CXLMemSim uses selective fidelity: it executes application computation natively and injects only the additional stalls caused by CXL memory. Three components cooperate. The Tracer records last-level-cache misses, allocation metadata, synchronization events, and branch context using PEBS, LBR, and eBPF probes. The Epoch Timer lets the program run for short intervals, then pauses it at safe points to charge accumulated CXL delay. The Timing Analyzer

maps sampled accesses to CXL pools, links, and switches, then computes latency, bandwidth, congestion, coherence, and policy effects.

*Profiling and attribution.* PEBS provides sampled instruction address, data address, latency, and access type for LLC misses without instrumenting every memory operation. eBPF probes track mmap, munmap, brk, page faults, stack growth, library mappings, and migration/remapping events, allowing each sampled address to be attributed to local DRAM or a CXL pool. A per-CPU attribution cache backed by a global radix tree serves 92–97% of lookups locally and keeps average attribution cost below 15 ns per sampled access.

*Latency, bandwidth, and synchronization.* Simply adding CXL latency to every miss overestimates slowdown because out-of-order cores hide 30–70% of memory latency when independent work is available. CXLMemSim combines L2-pending stall counters, L3-miss counters, and LBR-derived control-flow context with a light-weight ROB model. CXL misses become latency bubbles; only the uncovered tail that pushes the ROB frontier forward becomes visible delay. Bandwidth is modeled as a microsecond-granularity time series over each link and switch, with reads and writes accounted separately. For multi-threaded programs, lock, futex, condition-variable, and barrier probes become hard epoch boundaries, so delay accumulated inside a critical section is paid before unlock.

*Reusable release.* The open-source version is available at SlugLab/CXLMemSim. The release includes the Linux/x86 tracing path, JSON topology files for pools/switches/latencies/bandwidths, policy hooks for placement, migration, and caching, and reference policies including frequency promotion, locality-aware cluster migration, LRU/frequency-aware replacement, write-through/write-back caching, and a hybrid adaptive policy. This directly supports reusable CXL.mem studies before broad hardware access.

### 3 Evaluation

We evaluate on an Intel Xeon Gold 6787P server with 256 GB DDR5-8000 and an Intel Agilex 7 I-series FPGA implementing CXL.mem with programmable read/write latency from 50–500 ns and bandwidth throttling from 5–50 GB/s. Workloads cover SPEC CPU2017, Memcached, wrf, vector search, and llama.cpp, spanning irregular pointer chasing, mixed read/write key-value access, streaming arrays, batched search, and LLM weight/attention access. CXLMemSim is compared against native DRAM, the FPGA CXL device, and gem5 O3CPU with matched Xeon parameters.

CXLMemSim tracks hardware and gem5 accurately enough for system decisions. Load/store throughput and bandwidth match real CXL within 3–5% beyond LLC-sized working sets and within 8% for tiny working sets where prefetchers are unusually effective. Streaming latency error is below 8%, random access error is 8–11%, and pointer chasing is 12–15% because serialized dependencies amplify per-access queuing error. IPC and stall-cycle breakdowns remain within 7–10% of gem5; total multi-thread IPC error is under 10% from 2–16 threads. For Memcached at 8 threads, disabling synchronization-aware epochs raises IPC error from 8% to 23%, showing why delay must be paid before synchronization release. Switch modeling captures the 40–70 ns latency added by

each switch level within 10–12% and preserves the correct ranking between 25 GB/s and 50 GB/s links.

### 4 Policy Findings

CXLMemSim reveals that CXL management is workload and topology dependent. Frequency-based migration works well for stable hot sets: Redis and mcf improve by 15% and 17%, respectively, and stable workloads see up to 19%. It can waste work under phase changes, as in wrf, where it performs 3.2× more migrations than locality migration for only 2% more speed. Locality-aware cluster migration is better for spatially coherent AI accesses, improving llama.cpp by 18–19%, but can over-migrate pointer-heavy code. Recency migration is a robust default, usually within 5% of the best static policy.

Local DRAM as a CXL cache is equally sensitive. With local memory limited to 20% of the working set, write-back caching improves database workloads by 18–24% by absorbing bursts that would otherwise saturate 25 GB/s CXL links; at 10% local capacity the write-back advantage reaches 28% for Memcached, falls to 8% at 50%, and mostly disappears above 70%. Frequency-aware replacement improves hit rates by 12–15% over LRU when cold scans compete with long-lived hot structures. Topology also matters: a second switch level slows mcf by 22% but wrf by only 9%, and doubling link bandwidth from 25 to 50 GB/s yields 35–40% gains for many database and AI workloads. Overall, topology, bandwidth, migration, caching, and prefetching must be co-designed rather than tuned independently.

### 5 Scope and Conclusion

CXLMemSim has four main limitations. PEBS sampling can miss phases shorter than a few hundred microseconds. Native execution means caches, TLBs, and prefetchers are trained by local DRAM timing, so secondary CXL effects may be underestimated. Multi-thread modeling captures synchronization-visible delay but only approximates cache-line bouncing, memory-controller queuing, and extreme many-core contention, where bandwidth error can reach 10–15%. Finally, the current focus is CXL.mem, not CXL.cache or CXL.io interference.

CXLMemSim shows that practical CXL 3.0 simulation can be fast and accurate by pairing native execution with hardware-assisted profiling and microarchitecturally aware delay injection. The result is 123× speedup over gem5 while staying within 11–15% of measured behavior. By releasing CXLMemSim openly at SlugLab/CXLMemSim, we provide a reusable foundation for CXL topology, bandwidth, migration, and caching studies.

### References

- [1] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K. Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R. Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. 2011. The gem5 Simulator. In *Proceedings of the 1st ACM SIGARCH/CA/SPAA Workshop on Computer Architecture Research with gem5*.
- [2] Ionuț Calciu, Onur Mutlu, et al. 2021. Rethinking Software Runtimes for Disaggregated Memory. In *Proceedings of the 26th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21)*. ACM, 1077–1092.
- [3] Ahyoung Cho, Ali Daglis, et al. 2024. StarNUMA: Mitigating NUMA Challenges with Memory Disaggregation. In *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO '24)*.

- [4] CXL Consortium. 2020. Compute Express Link Specification Revision 2.0. <https://computeexpresslink.org/>. (2020). Accessed 2025.
- [5] Raja Gond and Purushottam Kulkarni. 2024. emuxl: an emulation framework for CXL-based disaggregated memory applications. *arXiv preprint arXiv:2404.08311* (2024).
- [6] Zhenyuan Guo, Huazhong Sun, Jinglei Ren, and Yiyang Zhang. 2022. Clio: A Hardware-Software Co-Designed Disaggregated Memory System. In *Proceedings of the 27th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, 417–433.
- [7] Ahsan Javed, Shivaram Venkataraman, et al. 2023. Design Tradeoffs in CXL-Based Memory Pools for Public Clouds. In *IEEE Micro*.
- [8] Seongbeom Kim, Jintaek Kang, Kyunghoon Kim, Seungwook Lee, and Beomseok Nam. 2025. CXLSim: A Simulator for CXL Memory Expander. In *2025 IEEE International Conference on Big Data and Smart Computing (BigComp)*. IEEE, 156–159.
- [9] Hao Li et al. 2024. Reconstructing RDMA-Based Memory Disaggregation via Rcmp. *Commun. ACM* (2024).
- [10] Huaicheng Li, Daniel S Berger, Stanko Novakovic, Lisa Hsu, Dan Ernst, Pantea Zardoshti, Monish Shah, Samir Rajadnya, Scott Lee, Ishwar Agarwal, et al. 2023. Pond: Cxl-based memory pooling systems for cloud platforms. In *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst.*
- [11] Rambus. 2024. Compute Express Link (CXL): All You Need to Know. <https://www.rambus.com/blogs/compute-express-link/>. (2024).
- [12] Jiaqi Zhang et al. 2024. DmRPC: Disaggregated Memory-aware Datacenter RPC for Microservices. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE '24)*.